

1 Introduction

In this homework assignment, you will use C++ to create and manage geometry which you will then pass to SFML for drawing lines. Managing geometry is a fundamental task in any graphics application. For this assignment, you will generate all required geometry using simple transformation rules. In particular, you will draw a space-filling curve known as the *Hilbert curve*. The construction rules for this fractal curve are explained in more detail later.

SFML provides different ways to draw lines. For this assignment, you will find *vertex arrays* and the primitives `Lines` and `LineStrip` helpful. Minimal starter code is provided in the course `sfml-starter` repository. Other details regarding transformations and handling interactions will be covered during tutorials.

1.1 Hilbert Curve

Discrete approximations to the 2D Hilbert curve are used to map data from a 2D space (e.g. an image) to a 1D index space. The good thing about this mapping is that it preserves locality—points that are close in 2D space are also close in index space thus leading to better cache locality when storing 2D data in memory. Fig. 1 shows an example of a *discrete* Hilbert curve on a 2D grid.

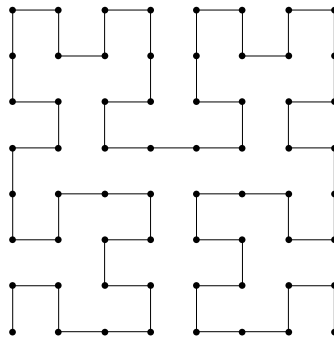


Figure 1: A discrete Hilbert curve on an 8×8 square grid.

1.2 Construction

There are different ways to generate a discrete Hilbert curve; for this assignment we will focus on a recursive geometric approach. Given a $2^n \times 2^n$ grid ($n \geq 1$) within the unit square $[0, 1]^2$, a Hilbert curve that visits all 2^{2n} points on the grid can be obtained by starting with a Hilbert curve defined on a $2^{n-1} \times 2^{n-1}$ grid and refining it by applying appropriate affine transformations. Fig. 2 illustrates this idea. The base case ($n = 1$) consists of the corners of the unit square that are visited in the order: $(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (1,0)$. In order to obtain the curve at the next level ($n = 2$), we scale down the the curve from the previous level ($n = 1$) and place four copies of it on the grid; two copies go to the top-left and top right while two *rotated* copies go to the bottom-left and bottom-right. We can then simply connect these copies together as shown in Fig. 2. This process can be repeated to obtain subsequent levels. In the limit (as $n \rightarrow \infty$), the curve obtained is the continuous Hilbert curve that fills the unit square.

The exact transformations that you need are not spelled out here but can be inferred from Fig. 2.

2 Requirements

Your task is to write a C++ program that procedurally generates the geometry associated with the Hilbert curve. Your program should then use SFML to draw the geometry on the screen. The number of iterations should be a user-adjustable parameter. Your program should be able to—through a keyboard control—produce and display the Hilbert curve at progressively increasing levels of detail.

2.1 Functional Requirements

Your program should have the following functionality:

- Generate and display the line segments associated with the discrete Hilbert curve for $n = 3$. (8 points)

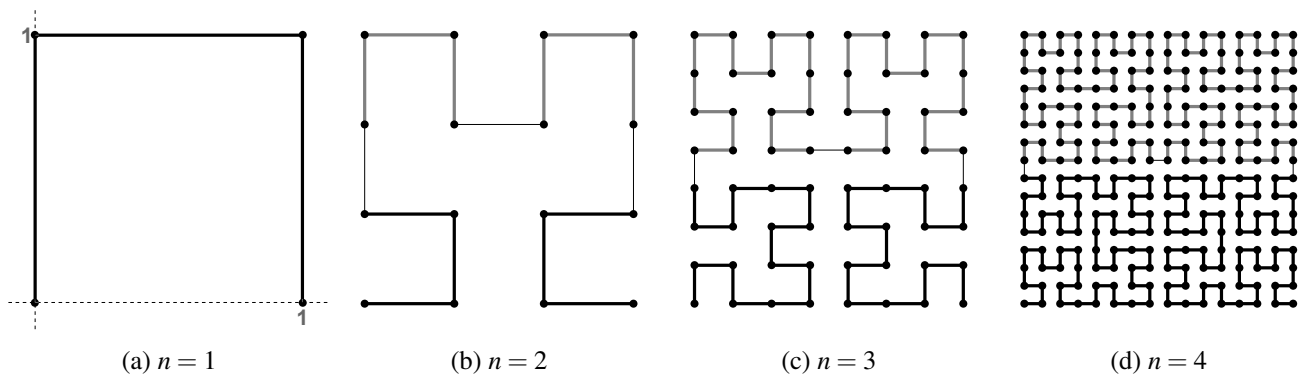


Figure 2: The first four iterations of the Hilbert curve. Each iteration consists of four scaled-down versions of the previous iteration.

- Provide keyboard controls that allow the user to display the curve at different levels of detail, starting from the level ($n = 3$), down to the base level $n = 1$ and up to the maximum level that the window can support. The user should be able to increase/decrease the level one step at a time.

To keep things simple, for this assignment, we'll be using a fixed window of size 1024×1024 . (12 points)

2.2 Non-Functional Requirements

Please also keep the following non-functional requirements in mind:

- You are required to build your program on top of the starter code provided in the sfml-starter repository.
- Please use the functionalities provided by SFML to manage geometry and handle transformations.
- Please do not change the build environment. Your program should compile via `cmake` just like the starter code.
- Please consult the official course outline to understand the policies on collaboration and the use of AI tools. To encourage responsible learning, the starter code includes a file named `DECLARATION AND REFLECTION`. You must complete it and submit it along with your work. (2 points)

2.3 Demo (5 points + 5 bonus)

You will be required to demo your program to your TA during a scheduled tutorial session in MS 239. Please be prepared to walk your TA through your implementation during the demo.

You can earn bonus points by demoing early.

3 Submission and Grading

Please submit via Gradescope. Please take note of the following when preparing your submission.

- Please submit your code and declaration as a self-contained package/repository that can be built/compiled on its own. Also, do not include any SFML-related files, since they are automatically fetched by the build process in the provided starter code.

4 Further Reading

- Hilbert curve on Wikipedia:
https://en.wikipedia.org/wiki/Hilbert_curve.
- Hilbert curve on Wolfram MathWorld:
<http://mathworld.wolfram.com/HilbertCurve.html>.